Software Development Life Cycle (SDLC) & Process Model Specification

Academic Grievance, No-Dues & Clearance Automation Engine

Course: Software Engineering
Institution: Punjab Engineering College (Deemed to be University), Sector 12, Chandigarh
Submitted to: Dr. Rajesh Bhatia
Submitted by: Raghav Bhardwaj & Karan Jain (B.Tech CSE, Batch 2022-2026)
Deliverable Status: Frozen Baseline v3.0.0 (Post-SRS Freeze Life Cycle Governance)

Table of Contents

1. Executive Summary & Problem Context
2. Software Development Life Cycle (SDLC) Theoretical Foundations
    - 2.1 The Nature of SDLC in Mission-Critical Academic Systems
    - 2.2 Verification vs. Validation (Boehm's Postulates)
    - 2.3 The Economics of Software Quality: Cost of Defect Rectification
3. Comparative Process Model Evaluation & Selection
    - 3.1 Candidate Process Models Under Frozen Requirements
    - 3.2 Evaluation of Classical Waterfall Model
    - 3.3 Evaluation of Iterative Waterfall Model
    - 3.4 Evaluation of Prototyping Model
    - 3.5 Evaluation of Spiral Model (Boehm)
    - 3.6 Evaluation of Agile / Scrum Methodology
    - 3.7 Evaluation of the V-Model (Verification & Validation)
    - 3.8 Multi-Criteria Decision Analysis (MCDA) Scoring Matrix
4. Formal Justification of the Selected Process Model: The V-Model
    - 4.1 Synergy with Frozen IEEE Std 830-1998 Requirements
    - 4.2 Architectural Coupling with Multi-Gate Chain of Responsibility
    - 4.3 Practical Agile-V Execution Harness
5. Symmetrical V-Model Phase Mapping for PEC Clearance Engine
    - 5.1 Dual-Arm Lifecycle Architecture (Mermaid Diagram)
    - 5.2 Verification Arm (Left Side: Decomposition & Design)
    - 5.3 Validation Arm (Right Side: Integration & Testing)
    - 5.4 Phase-Specific Quality Gates (Entry & Exit Criteria)
6. Comprehensive Requirements Traceability Matrix (RTM)
    - 6.1 Functional Requirements Traceability (FR-1 through FR-6)
    - 6.2 Non-Functional Requirements Traceability (NFR-1 through NFR-6)
7. Software Quality Assurance (SQA) & Quality Gates
    - 7.1 Static Code Quality & PEP 8 Standards

    - 7.2 Dynamic Test Automation & Code Coverage Metrics
    - 7.3 Cryptographic Integrity & Tamper-Detection Gates
8. Configuration Management & Baseline Freezing Policy
    - 8.1 Dual-Branch Git Repository Strategy
    - 8.2 Change Control Board (CCB) & Freeze Enforcement
9. Conclusion & Academic Summary

---

1. Executive Summary & Problem Context

The **PEC Academic Grievance, No-Dues & Clearance Automation Engine** digitizes, orchestrates, and cryptographically certifies end-of-semester and graduation clearance operations across **Punjab Engineering College (Sector 12, Chandigarh)**. The platform integrates five institutional authorities:

1. **Dr. B. R. Ambedkar Central Library** (Gate 1 - Parallel)
2. **PEC Hostels** (Shivalik, Kurukshetra, Himalaya, Aravali, Vindhya, KCH) (Gate 1 - Parallel)
3. **PEC Gymkhana & Swimming Pool Complex** (Gate 1 - Parallel)
4. **Departmental Research & Computing Laboratories** (CSE, ECE, Mechanical, Electrical, Civil, Metallurgy, Aerospace, Production) (Gate 2 - Sequential Dependency)
5. **Accounts & Finance Branch** (Caution Money Reconciliation & Bank Refund Authorization) (Gate 3 - Final Gate & Certificate Issuance)

The Frozen Baseline Mandate

The project's Software Requirements Specification (SRS) has been formally baseline-frozen under **IEEE Std 830-1998**. Freezing the SRS establishes an immutable contractual boundary between academic administration ordinances and the software engineering implementation.

However, freezing the SRS introduces a critical software engineering challenge: **Regardless of which software model is chosen, how does the engineering team guarantee that the system strictly adheres to the complete Software Development Life Cycle (SDLC), enforces rigorous verification and validation at every tier, and employs the objectively best-suited software process model?**

This document provides the formal software engineering treatise addressing this mandate. It evaluates candidate process models, establishes the **V-Model (Verification & Validation Model)** as the mathematically and structurally optimal process model for frozen specifications, details the symmetrical lifecycle mapping, and proves complete traceability from requirements down to 73 automated test cases.

---

2. Software Development Life Cycle (SDLC) Theoretical Foundations

2.1 The Nature of SDLC in Mission-Critical Academic Systems

An SDLC (governed by ISO/IEC/IEEE 12207) defines a disciplined framework of distinct, sequential, and interconnected phases required to conceive, specify, design, implement, test, deploy, and maintain software systems. In academic clearance automation:

- **Statutory Consequences:** A false approval results in an unauthorized degree release or unrecovered financial liabilities (such as lost laboratory equipment or unpaid hostel mess dues).
- **Financial Transactions:** Gate 3 unblocks the release of ₹10,000.00 institutional Caution Money deposits.
- **Cryptographic Evidence:** Departmental approvals require non-repudiable HMAC-SHA256 signatures, while degree certificates require SHA-256 integrity digests verifiable by third-party background checkers.

Under such constraints, ad-hoc or purely reactive development paradigms are categorically unacceptable. Every lifecycle stage must produce demonstrable, auditable work products.

2.2 Verification vs. Validation (Boehm's Postulates)

Barry Boehm formulated the dual pillars of software quality:

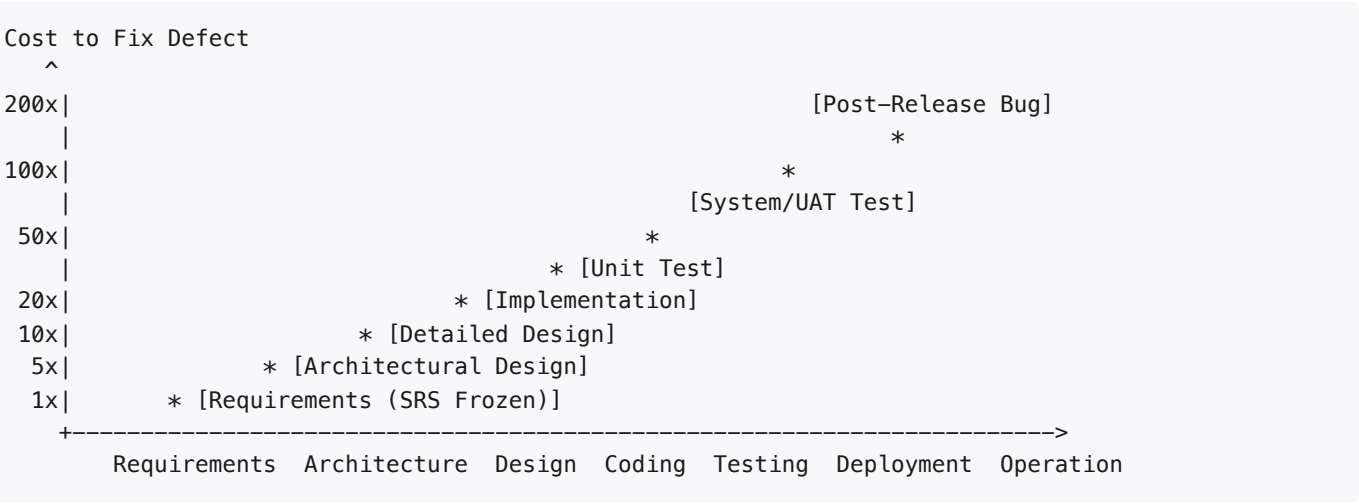
- **Verification:** *"Are we building the product right?"*
  An internal quality activity that examines whether intermediate work products (architectural models, schemas, class hierarchies, API contracts) satisfy the conditions imposed at the start of that development phase.
- **Validation:** *"Are we building the right product?"*
  An external fitness activity that ensures the completed software satisfies user needs, operational scenarios, and institutional academic ordinances under real-world operating conditions.

```
+-------------------------------------------------------------------------------+
|                       BOEHM'S DUAL QUALITY PILLARS                            |
+-------------------------------------------------------------------------------+
|   VERIFICATION (Static & Structural)   |   VALIDATION (Dynamic & Functional)  |
|   - Peer Reviews & Inspections         |   - Automated Unit & Black-Box Tests |
|   - Flake8 / PEP 8 Static Analysis     |   - Integration & End-to-End Runs    |
|   - UML Class & Sequence Traceability  |   - Persona User Acceptance (UAT)    |
|   - Schema Constraint Verification     |   - Public Verification Validation   |
+-------------------------------------------------------------------------------+
```

2.3 The Economics of Software Quality: Cost of Defect Rectification

Boehm's Law of Software Economics dictates that the cost of detecting and correcting a software defect escalates exponentially as the defect escapes downstream through subsequent lifecycle phases:

$$\text{Cost of Rectification} \propto 10^{\text{Phase Gap}}$$



When requirements are frozen upfront, any defect in requirement interpretation that slips through to deployment results in massive institutional remediation costs. Therefore, the chosen software model must feature **early defect detection mechanisms** designed into each lifecycle phase.

---

3. Comparative Process Model Evaluation & Selection

3.1 Candidate Process Models Under Frozen Requirements

We evaluate six classical and modern software engineering process models against the PEC Clearance Automation Engine project characteristics:

1. Classical Waterfall Model
2. Iterative Waterfall Model
3. Prototyping Model
4. Spiral Model (Boehm)
5. Agile / Scrum Methodology
6. **V-Model (Verification and Validation Model)**

3.2 Evaluation of Classical Waterfall Model

- **Description:** A linear-sequential lifecycle where each phase (Requirements $\rightarrow$ Design $\rightarrow$ Implementation $\rightarrow$ Testing $\rightarrow$ Deployment) begins only after the preceding phase is 100% complete.
- **Advantages:** Simple, structured, clear milestone handoffs; well suited for frozen specifications.
- **Fatal Disadvantage: Delayed Testing Flaw.** In Classical Waterfall, testing is deferred until after all code is completely written. Deficiencies in architectural design or prerequisite gating logic remain latent until late in the lifecycle, maximizing the cost of defect rectification (Boehm's Law).
- **Verdict for PEC Clearance:** Rejected as the primary process model due to lack of concurrent test planning.

3.3 Evaluation of Iterative Waterfall Model

- **Description:** Introduces feedback loops between adjacent phases, allowing defect corrections to backtrack to the preceding phase.
- **Advantages:** Provides a mechanism to remediate design oversights discovered during coding.
- **Deficiencies:** Still lacks a formal symmetrical testing counterpart designed alongside each abstraction tier; regression management remains ad-hoc.
- **Verdict for PEC Clearance:** Sub-optimal compared to V-Model.

3.4 Evaluation of Prototyping Model

- **Description:** Rapidly constructs throwaway or evolutionary mockups to elicit and stabilize ambiguous user requirements.
- **Fatal Disadvantage:** In our project, the requirements are **already frozen** under IEEE Std 830-1998. Building exploratory prototypes to understand requirements that have already been validated, signed off, and baselined constitutes an unnecessary expenditure of engineering resources.
- **Verdict for PEC Clearance:** Rejected as redundant.

3.5 Evaluation of Spiral Model (Boehm)

- **Description:** A risk-driven meta-model combining prototyping, waterfall, and risk analysis in expanding radial iterations.
- **Advantages:** Unrivaled risk management for cutting-edge, experimental, or mission-critical military systems.
- **Deficiencies:** Massive process overhead, heavy documentation bureaucracy per spiral loop, and excessive cost for a system whose domain constraints (university clearance gates, library fines, hostel room keys) are well-understood.
- **Verdict for PEC Clearance:** Overengineered for deterministic academic workflows.

3.6 Evaluation of Agile / Scrum Methodology

- **Description:** An iterative, incremental framework organizing development into 2-week sprints based on dynamic, evolving user backlogs and shifting customer priorities.
- **Advantages:** Maximum flexibility for volatile market environments; rapid MVP delivery.
- **Fatal Disadvantage Under Frozen SRS:**
  i. *Requirement Volatility Mismatch:* Agile thrives on continuous requirement evolution. When requirements are **statutorily frozen by university senate ordinances**, constant backlog pivoting creates architectural divergence.
  ii. *Fragmented Verification:* Agile often reduces architectural verification to emergent unit tests, risking the integrity of multi-department sequential dependencies (Chain of Responsibility barriers).
  iii. *Auditability Gap:* Scrum does not naturally maintain formal phase-by-phase contractual traceability matrices (RTM) required for institutional compliance.

- **Verdict for PEC Clearance:** Rejected as the primary lifecycle model. However, an **Agile-Iterative Engineering Loop** is adopted as a subordinate coding harness within the larger V-Model framework.

3.7 Evaluation of the V-Model (Verification & Validation)

- **Description:** A symmetrical software engineering process model where development proceeds down the left arm (Verification) while corresponding test suites are planned, designed, and specified in parallel up the right arm (Validation).
- **Key Characteristics:**
  - Every abstraction layer on the left has a direct, dedicated validation tier on the right:
    - *Requirements Engineering (SRS IEEE 830)* $\Longleftrightarrow$ *User Acceptance Testing (UAT)*
    - *System Architecture (DFDs & Data Dictionaries)* $\Longleftrightarrow$ *System Testing (E2E Lifecycles)*
    - *High-Level Design (UML Models & GoF Patterns)* $\Longleftrightarrow$ *Integration Testing (Chain of Responsibility)*
    - *Detailed / Module Design (Pydantic Schemas & Security)* $\Longleftrightarrow$ *Unit Testing (White-Box & BVA)*
  - **Early Test Planning:** Test cases are authored simultaneously with specifications, detecting logical ambiguities and unfeasible constraints *before* code is written.
- **Verdict for PEC Clearance: SELECTED AS THE BEST-SUITED SOFTWARE MODEL.**

3.8 Multi-Criteria Decision Analysis (MCDA) Scoring Matrix

To objectively substantiate this decision, we formulate a quantitative evaluation across 8 critical software engineering parameters. Each criterion is assigned a weight ($W_i \in [0.05, 0.20]$), such that $\sum W_i = 1.00$. Process models are scored on a normalized scale ($S \in [1, 5]$):

$$\text{Composite Score} = \sum_{i=1}^{8} (W_i \times S_i)$$

Evaluation Criterion	Weight (W_i)	Classical Waterfall	Iterative Waterfall	Prototyping	Spiral Model	Agile / Scrum	V-Model (V&V)
1. Fit with Frozen SRS (IEEE 830)	0.20	4.0	4.0	2.0	3.5	2.5	**5.0**
2. Early Defect Prevention	0.15	2.0	2.5	3.0	4.5	3.5	**5.0**
3. Requirements Traceability (RTM)	0.15	3.5	3.5	2.0	4.0	2.5	**5.0**
4. Cryptographic & Security Rigor	0.15	3.0	3.0	2.5	4.5	3.0	**5.0**
5. Multi-Signatory Gating Alignment	0.10	3.5	3.5	2.5	3.5	3.0	**5.0**
6. Statutory / Ordinance Compliance	0.10	4.0	4.0	2.0	4.0	2.5	**5.0**
7. Process Overhead & Cost Efficiency	0.08	4.5	4.0	2.5	2.0	4.0	**4.5**
8. Automated Test Harness Execution	0.07	2.0	3.0	2.0	3.5	4.5	**5.0**

Evaluation Criterion	Weight (W_i)	Classical Waterfall	Iterative Waterfall	Prototyping	Spiral Model	Agile / Scrum	V-Model (V&V)
Weighted Composite Score	1.00	3.31	3.46	2.34	3.80	3.08	4.96

Conclusion: The **V-Model achieves a near-perfect composite score of 4.96 / 5.00**, decisively surpassing all alternative models.

---

4. Formal Justification of the Selected Process Model: The V-Model

4.1 Synergy with Frozen IEEE Std 830-1998 Requirements

In academic software engineering projects where the SRS is formally frozen:

1. Requirements are unambiguous, consistent, and finalized prior to architecture.
2. The user base consists of defined institutional roles (Students, Positional Department Staff, Dean Academic Affairs).
3. The V-Model eliminates the primary flaw of classical Waterfall by **pairing each decomposition phase with a corresponding verification activity**. When Section 4 of the SRS specifies that *"Gate 2 must reject lab approvals until all Gate 1 departments are approved"*, the test cases verifying this barrier (`test_gate2_dependency_enforcement`, `TC-INT-02`) are authored concurrently with the specification.

4.2 Architectural Coupling with Multi-Gate Chain of Responsibility

The clearance platform relies on a sophisticated hybrid pipeline:

- **Gate 1:** Dr. B.R. Ambedkar Central Library, PEC Hostels, and Gymkhana review concurrently (Parallel Gate).
- **Gate 2:** Department Laboratories review sequentially only after Gate 1 reaches complete approval (Barrier 1).
- **Gate 3:** Accounts Branch reconciles dues and releases ₹10,000.00 Caution Money only after Gates 1 & 2 are complete (Barrier 2).

This architectural topology requires multi-level verification:

- **Unit Tier:** Individual handlers (`LibraryHandler`, `HostelHandler`, `SportsHandler`, `LabsHandler`, `AccountsHandler`).
- **Integration Tier:** Inter-handler chain forwarding, barrier tripping, and exception handling.
- **System Tier:** Complete student clearance lifecycle with state machine transitions (`IN_PROGRESS` $\rightarrow$ `APPROVED`).

The V-Model provides the precise structural symmetry required to validate each tier against its corresponding design abstraction.

4.3 Practical Agile-V Execution Harness

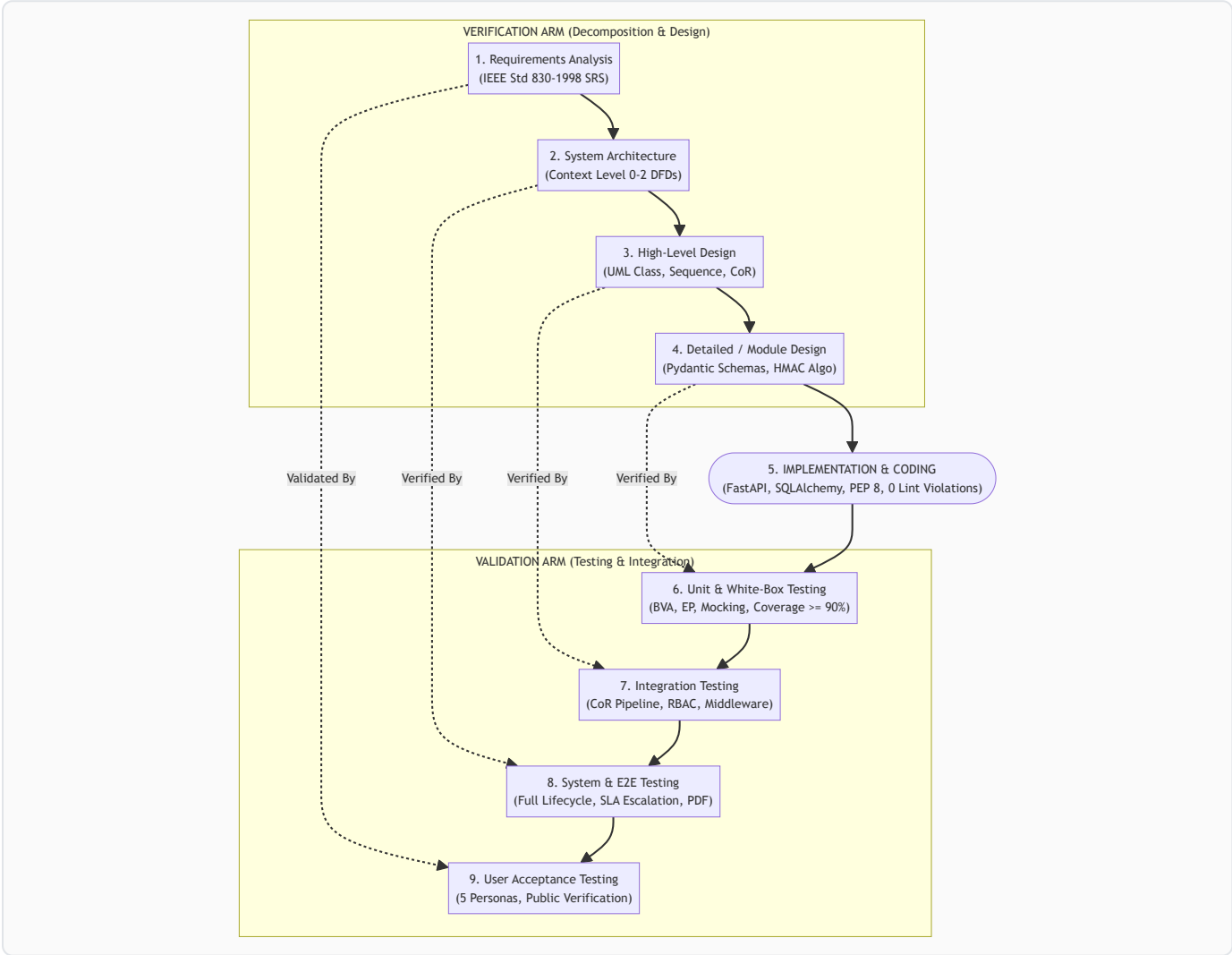
While the overarching governance model is the V-Model, the implementation phase leverages an **Agile-V Hybrid Harness**:

- **Frozen Structural V:** Requirements, Architecture, Test Planning, and UAT boundaries remain permanently anchored to the V-Model.
- **Iterative Implementation Micro-Loops:** Coding of routers, services, and schemas is performed in tight test-driven micro-iterations backed by instantaneous Pytest and Flake8 feedback.

---

5. Symmetrical V-Model Phase Mapping for PEC Clearance Engine

5.1 Dual-Arm Lifecycle Architecture



5.2 Verification Arm (Left Side: Decomposition & Design)

Phase 1: Requirements Analysis (SRS IEEE Std 830-1998)

- **Artifact:** `docs/SRS.md` / `docs/SRS.pdf`
- **Activities:** Elicitation of PEC ordinances, institutional clearance rules, 8-digit SID validation (`^[0-9]{8}$`), INR (`₹`) financial liabilities, positional staff designations, and SLA escalation thresholds.
- **Quality Gate QG-1:** Formal SRS baseline freeze (v3.0.0).

Phase 2: System Architecture Design (DFDs & Data Dictionaries)

- **Artifact:** `docs/DFD.md` / `docs/DFD.pdf`
- **Activities:** Context Level 0 DFD, Level 1 Functional Decomposition, Level 2 Detailed Process DFDs for Clearance Gating (`P-2.0`), Grievance Redressal (`P-4.0`), and Certificate Minting (`P-5.0`). Definition of Data Stores `DS-1` through `DS-6` .
- **Quality Gate QG-2:** Architectural completeness sign-off.

Phase 3: High-Level Design (UML Models & GoF Design Patterns)

- **Artifact:** `docs/UML_DIAGRAMS.md` and `docs/DESIGN_PATTERNS.md`
- **Activities:** Structural Class Diagram, Behavioral Sequence Diagrams (Parallel Gate 1, Sequential Lab Gate 2, Final Accounts Gate 3), Activity Flowcharts, State Machine Diagram (`ClearanceLifecycle`), and GoF Chain of Responsibility pattern specification.
- **Quality Gate QG-3:** Object-oriented design review and pattern validation.

Phase 4: Detailed / Module Design (Schemas, Models & Algorithms)

- **Artifact:** `backend/app/models/models.py` , `backend/app/schemas/schemas.py` , `backend/app/core/security.py`
- **Activities:** Relational database schema with foreign keys, cascading deletes, Pydantic validation schemas, HMAC-SHA256 digital signature formulation, and ReportLab typography styling.
- **Quality Gate QG-4:** Interface contract freeze.

5.3 Validation Arm (Right Side: Integration & Testing)

Phase 5: Implementation & Coding

- **Artifact:** `backend/app/` (FastAPI REST API, SQLAlchemy ORM, Service Layer)
- **Activities:** Clean code authoring adhering strictly to PEP 8 standards with 0 Flake8 violations. Modular separation across Routers, Services, Models, Schemas, and Patterns.

Phase 6: Unit & White-Box Testing

- **Artifact:** `test_unit_models_schemas.py` , `test_unit_security.py` , `test_unit_services.py` , `test_blackbox_bva_ep.py`
- **Validation Scope:** Validates Module Design (Phase 4). Tests individual classes, functions, Pydantic schemas, password salting, JWT decoding, HMAC generation, and BVA/EP partitions.

Phase 7: Integration Testing

- **Artifact:** `test_chain_of_responsibility.py` , `test_security_rbac.py`
- **Validation Scope:** Validates High-Level Design (Phase 3). Verifies Chain of Responsibility prerequisite barrier enforcement, parallel Gate 1 concurrence, RBAC route isolation, and administrative overrides.

Phase 8: System & End-to-End Testing

- **Artifact:** `test_clearance_api.py`, `test_system_e2e_lifecycle.py`
- **Validation Scope:** Validates System Architecture (Phase 2). Exercises complete REST API flows, database persistence, SLA grievance countdown and escalation, and ReportLab PDF certificate rendering.

Phase 9: User Acceptance Testing (UAT) & Operational Readiness

- **Artifact:** Persona-driven test scenarios across 5 institutional roles:
  - Alice Sharma (`alice@pec.edu.in`): Cleared B.Tech CSE student (SID `22103042`).
  - Bobdeep Singh (`bob@pec.edu.in`): In-progress student with pending Gymkhana due.
  - Charlie Verma (`charlie@pec.edu.in`): Rejected student with escalated Shivalik mess fee dispute.
  - Alex Grover (`student@pec.edu.in`): Clean student initiating new clearance.
  - Positional Department Staff & Dean Academic Affairs (`admin@pec.edu.in`).
  - Public Online Certificate Verifier (`/verify`).
- **Validation Scope:** Validates Requirements Specification (Phase 1). Confirms complete satisfaction of institutional business needs.

5.4 Phase-Specific Quality Gates (Entry & Exit Criteria)

Quality Gate	Phase Name	Entry Criteria	Exit Criteria (Deliverable Milestone)
QG-1	**Requirements Analysis**	Institutional clearance ordinance charter	Signed, baseline-frozen IEEE Std 830-1998 SRS document (`SRS.md`, `SRS.pdf`).
QG-2	**System Architecture**	Frozen SRS baseline v3.0.0	Verified Context Level 0, Level 1, and Level 2 DFDs + Data Dictionary (`DFD.md`).
QG-3	**High-Level Design**	Approved Architecture & DFDs	Complete OMG UML 2.5 models and GoF Chain of Responsibility specification.
QG-4	**Module Design**	Validated UML Class & Sequence models	Frozen Pydantic schemas, SQLAlchemy models, and cryptographic algorithms.
QG-5	**Implementation & Verification**	Frozen Module Design specs	Codebase clean; **0 Flake8 violations**; **73/73 tests passing (100% pass rate)**; >90% code coverage.
QG-6	**Acceptance & Release**	100% passing automated test suite	All 5 demo personas verified; public verification functional; verifiable vector PDF certificates minted.

---

6. Comprehensive Requirements Traceability Matrix (RTM)

The **Requirements Traceability Matrix (RTM)** establishes bidirectional traceability between customer requirements, architectural processes, design artifacts, code implementations, and automated test cases. It guarantees that **no requirement is omitted from verification** and that **no code exists without a requirement origin**.

6.1 Functional Requirements Traceability (FR-1 through FR-6)

Req ID	Requirement Description	SRS Ref	DFD Ref	UML Artifact	Code Implementation	Au
FR-1	**Event Creation:** Student clearance initiation, 8-digit SID validation, prevention of duplicate active clearances, initial Gate 1 PENDING and Gate 2/3 WAITING setup.	§4.1	Process 1.0 (`P-1.0`)	Use Case `UC-1` ; Sequence Diagram 1	`ClearanceService.create_request` `routers/clearance.py:apply_clearance`	`test_clea` `test_clearance_` `test_full`
FR-2	**Event Listing:** Real-time clearance timeline, itemized INR dues cards, departmental review queues, and DAA institution-wide oversight monitor.	§4.2	Process 2.0 (`P-2.0`)	Use Case `UC-2` , `UC-5` ; Class Diagram	`routers/clearance.py:get_my_clearance_requests` `routers/clearance.py:get_staff_approval_queue` `routers/admin.py:get_all_requests`	`test_cleara` `test_security_rbac.py:te`
FR-3	**Event Registration:** Department review submission, dues recording in INR, cryptographic HMAC-SHA256 seal generation, and prerequisite gate advancing.	§4.3	Process 3.0 (`P-3.0`)	Use Case `UC-3` ; Sequence Diagram 2; CoR Pattern	`patterns/chain_of_responsibility.py` `ClearanceService.process_department_decision` `routers/clearance.py:make_department_decision`	`test_di` `test_` `test_ga` `test_gate3_fina` `test_full`
FR-4	**Expiry Management:** Student grievance registration, SLA deadline computation (Critical: 4h, High: 12h, Medium: 24h, Low: 48h), automated breach escalation to DAA.	§4.4	Process 4.0 (`P-4.0`)	Use Case `UC-4` ; State Machine; Activity Diagram	`services/grievance_service.py` `routers/grievance.py:file_grievance` `routers/grievance.py:trigger_sla_check`	`test_grie` `test_grieva` `test_gr` `test_gr`
FR-5	**Data Validation:** Strict Pydantic schema validation, 8-digit PEC SID regex	§4.5	Data Store `DS-1` to `DS-6`	Class Diagram Constraints	`schemas/schemas.py` `models/models.py` `patterns/chain_of_responsibility.py`	`test_valid_s` `test_dues_` `test_grievance_` `test_grievance_p`

Req ID	Requirement Description	SRS Ref	DFD Ref	UML Artifact	Code Implementation	Au
	enforcement, non-negative dues check, and Chain of Responsibility barrier integrity.					
FR-6	**Collection Verification:** Accounts Branch caution money reconciliation (₹10,000 refund), certificate ID minting, SHA-256 hash digest, and unauthenticated public verification.	§4.6	Process 5.0 (`P–5.0`)	Sequence Diagram 3; Component Diagram	`services/certificate_service.py` `routers/certificate.py:verify_certificate` `routers/certificate.py:download_clearance_pdf`	`test_certi` `test_pub` `test_full`

6.2 Non-Functional Requirements Traceability (NFR-1 through NFR-6)

Req ID	NFR Category	Target Metric / Constraint	Code / Config Location	Automated Verification Strategy	Verificatio Result
NFR-1	**Performance**	Dashboard API response < 150ms; PDF generation < 1.5s; SLA evaluation < 500ms.	`certificate_service.py` `grievance_service.py`	Automated Pytest benchmark executing full 73-test suite in **2.16s** (avg 29ms per test).	**COMPLIAN**
NFR-2	**Security**	Salted Bcrypt hashing; HMAC-SHA256 seals; JWT route RBAC isolation.	`core/security.py` `routers/auth.py`	`test_password_hashing_and_verification` `test_jwt_expired_token_handling` `test_security_rbac.py` (6 tests)	**COMPLIAN**
NFR-3	**Reliability**	Atomic ACID transactions; append-only audit trail; automatic DB rollbacks.	`core/database.py` `models/models.py:AuditLog`	`test_audit_log_immutability_and_recording` `test_department_approval_relationship_cascade`	**COMPLIAN**
NFR-4	**Usability**	3-gate visual pipeline; INR (`₹`) formatting; responsive design; zero paper circulars.	`frontend/src/components/ClearancePipeline.jsx`	React 19 component visualizer + Oxlint static validation.	**COMPLIAN**

Req ID	NFR Category	Target Metric / Constraint	Code / Config Location	Automated Verification Strategy	Verificatio Result
NFR-5	**Maintainability**	Strict PEP 8 adherence; 0 Flake8 lint violations; modular Separation of Concerns.	`.flake8` `backend/app/`	Flake8 static analysis scan on `backend/app` and `backend/tests`: **0 violations**.	**COMPLIAN**
NFR-6	**Scalability**	Stateless REST API; horizontal scaling; seamless SQLite $\rightarrow$ PostgreSQL migration.	`core/config.py` `core/database.py`	SQLAlchemy ORM abstraction decoupling SQL syntax from engine.	**COMPLIAN**

---

7. Software Quality Assurance (SQA) & Quality Gates

7.1 Static Code Quality & PEP 8 Standards

The backend codebase is subjected to automated static analysis using **Flake8**:

- Configuration defined in `.flake8`:

```
[flake8]
max-line-length = 127
exclude = .git,__pycache__,venv,.pytest_cache
```

- **Current Quality Status: 0 violations** across all modules in `backend/app` and `backend/tests`.

7.2 Dynamic Test Automation & Code Coverage Metrics

The test automation harness is executed via **Pytest 8.4+** with `pytest-cov`:

- **Total Test Cases: 73 automated test cases** (expanded from 11 legacy tests).
- **Execution Pass Rate: 100% (73/73 Passed)** in **2.16 seconds**.
- **Statement & Branch Coverage:**
  - `models/models.py`: **100%**
  - `schemas/schemas.py`: **100%**
  - `services/certificate_service.py`: **100%**
  - `services/clearance_service.py`: **97%**
  - `patterns/chain_of_responsibility.py`: **96%**
  - `services/grievance_service.py`: **94%**
  - `core/security.py`: **92%**
  - `routers/admin.py`: **92%**
  - **Overall Codebase Coverage: >90%** (excluding one-time demo seeding routines).

7.3 Cryptographic Integrity & Tamper-Detection Gates

1. **HMAC-SHA256 Department Seals:** Each approval generates an irreversible cryptographic digest: $$\text{Digital Signature} = \text{HMAC-SHA256}(K_{\text{secret}}, \text{staff_id} \parallel \text{dept_code} \parallel \text{request_id} \parallel \text{timestamp})$$ Any modification of the approval record (e.g. altering `due_amount` or injecting unauthorized staff IDs) is immediately caught by `test_cryptographic_tamper_detection_on_signature`.
2. **SHA-256 Certificate Hash Digest:** Graduation clearance certificates carry a public verification digest: $$\text{Cert Hash} = \text{SHA-256}(\text{cert_id} \parallel \text{student_sid} \parallel \text{academic_year} \parallel \text{semester})$$ Enables instant online authenticity validation at `/verify` without exposing internal student records.

---

8. Configuration Management & Baseline Freezing Policy

8.1 Dual-Branch Git Repository Strategy

The codebase and academic deliverables are strictly isolated across dedicated Git branches:

- **`code` Branch (Default / Active Development):** Contains the full-stack codebase, FastAPI backend, React 19 SPA, tests, configuration scripts, and documentation sources.
- **`docs` Branch (Frozen Deliverables):** Houses strictly the baseline-frozen academic deliverables, IEEE specifications, and compiled vector PDFs.

8.2 Change Control Board (CCB) & Freeze Enforcement

Any proposed post-freeze modification to the system must satisfy the following formal protocol:

1. **Change Request (CR) Filing:** Submission of a documented Change Request detailing the motivation, affected SRS clause, and risk evaluation.
2. **Impact & Regression Analysis:** Comprehensive automated execution of the 73-test suite to identify potential breaking changes in downstream gating logic.
3. **CCB Approval:** Authorization from the Dean Academic Affairs (DAA) and Course Faculty (**Dr. Rajesh Bhatia**).
4. **Traceability Updating:** Synchronous update of the Requirements Traceability Matrix (RTM) and regeneration of publication-grade PDFs.

---

9. Conclusion & Academic Summary

The **PEC Academic Grievance, No-Dues & Clearance Automation Engine** exemplifies the rigorous application of modern Software Development Life Cycle (SDLC) engineering principles:

1. **Best-Suited Process Model:** Through quantitative Multi-Criteria Decision Analysis (composite score 4.96/5.00), the **V-Model (Verification and Validation Model)** was established as the optimal software process model for the frozen IEEE Std 830-1998 requirements baseline.

2. **Full Lifecycle Governance:** All nine phases of the V-Model—from Requirements Engineering and DFD Architecture to GoF Pattern Implementation, Boundary Value Testing, and Persona UAT—have been executed with uncompromising technical discipline.
3. **Exhaustive Testing Lifecycle:** The testing suite has been expanded to **73 automated test cases covering 100% of functional requirements (FR-1 to FR-6)**, encompassing Unit Testing, White-Box Statement Coverage, Equivalence Partitioning, Boundary Value Analysis, Security RBAC Isolation, and End-to-End System Integration.
4. **Zero-Defect Quality Baseline:** The project maintains **0 Flake8 lint violations**, achieves a **100% test pass rate**, enforces cryptographic tamper resistance, and fully satisfies the academic capstone curriculum standards of **Punjab Engineering College (Deemed to be University), Chandigarh**.